PRIFYSGOL
BANGOR
UNIVERSITY

ICE-2102: Application Development

Portfolio Assignment

Description

Your overall task for this project is to create a Point of Sale (POS) application for a small shop. At minimum the app should have a Graphical User Interface (GUI), and use data stored in a relational database housed on a server. It is up to you to explore what the requirements and options could be beyond this. In this module we are focusing on the process and steps taken in developing an app, rather than implementation specifics. However, if you are unsure about your app design, please ask the lecturer for advice.

You will be undertaking the development process over the course of the academic year. This will involve identifying a problem to solve, specifying and implementing a solution to that problem, and conducting tests to ensure the quality of the solution. Your work will be presented in a portfolio containing six (6) sections. Each section has a mark value, and two main milestone deadlines to help you keep on track.

Total number of marks: **70**, directly translating to 70% of this module.

Breakdown:

Portfolio Section	**Marks Available**
Problem Statement	5
Problem Breakdown Document	5
Program Specification (derived from the Problem Breakdown Document)	10
Pseudo Code	5
Program Code	40
Testing Plan for your own Program	5

Due Dates (All due 23:59):

- **Friday 20th December 2025**
    1. Problem Statement
    2. Problem Breakdown Document
    3. Program Specification

- **End of April 2026**
    1. Pseudo Code
    2. Program Code
    3. Test Plan

Guidance

- You will need to craft your final solution using C#. No other languages will be accepted.

- Marking will be focusing on attention to detail. Each Portfolio section has a description in this document on what this means.

Plagiarism & Unfair Practice

Plagiarised work will be given a mark of zero. Remember when you submit you agree to the standard agreement:

This piece of work is a result of my own work except where it is a group assignment for which approved collaboration has been granted. Material from the work of others (from a book, a journal or the Web) used in this assignment has been acknowledged and quotations and paraphrasing suitably indicated. I appreciate that to imply that such work is mine, could lead to a nil mark, failing the module or being excluded from the University. I also testify that no substantial part of this work has been previously submitted for assessment.

Late Submission

Work submitted within one week of the stated deadline will be marked but the mark will be capped at 40%. A mark of 0% will be awarded for any work submitted 1 week after the deadline. Students must follow the electronic extension request process of the University - if necessary.

Feedback

Students are encouraged to solicit formative feedback during scheduled laboratory times, by e-mail, or in person for each element prior to submission. In order for students to make the most of this feedback, they are also encouraged to request feedback at the earliest possible juncture. The assessors will attempt to return marks and written feedback on each element within 2 weeks of submission, however, we will keep you informed if it needs to extend to the 4 weeks allowed by the University. More detailed comments can be sought on a one-to-one basis by mutual agreement.

Submission Procedure

Written portions of this assessment must be submitted in Word or PDF format to the appropriate Blackboard assessment slot.

Code and other electronic portions must be submitted as a ZIP file **(no other archive formats will be accepted)** to the appropriate Blackboard assessment slot.

Section 1 - Problem Statement (5 marks)

This portion of your Portfolio focuses on the reasons that your application would be needed. Essentially, what problem are you solving? The problem needs to be specific enough to allow the reader to understand what sort of application will eventually get built, but not so specific that it may as well be the requirements document. It must also include reasons why an existing solution does not address the problem. These reasons may be completely hypothetical, but they need to be plausible. A good problem statement can express these points in two or so paragraphs, however this submission should be ***no more than*** one A4 side.

The Problem Statement does not explicitly set goals, as a requirements document should. However, it must provide hints as to what the requirements might be when fully fleshed out.

Mark Scheme

Mark	Criteria
0-1	The student has not formulated a plausible problem to address or not been able to reasonably articulate the problem.
2	The student has formed a coherent description of the problem to be addressed, but has not examined any motivations or alternatives.
3-4	The student articulated a clear description of the chosen problem including motivations. There is limited examination of existing alternatives.
5	The student has produced a clear definition of the problem, with motivations and examined all logical/plausible alternatives. The definition is clear and concise.

Section 2 - Problem Breakdown Document (5 marks)

The Problem Breakdown Document is the first step to planning your application. This document takes the general themes outlined in your Problem Statement and breaks them down into features. These features should be organised into groups. This document is very similar to the Work Breakdown Structure documents used in Project Planning.

The entire purpose is to create a scope, almost like a tick-list, for the development cycle. Once all of the items on the list are complete, the software is finished, or at least this version is. The document will be structured like an outline or table of contents. As you move to lower levels, the more detail will be added. This document does not decide *how* those features are implemented, but *what* goes into them.

This submission for this section should be ***no more than*** four A4 sides.

Mark Scheme

Mark	Criteria
0-1	The student has not been able to present a structure to solving the problem defined in their Problem Statement, the breakdown does not present the same problem.
2	The breakdown is related to the original problem and major features are identified, however the definition is too broad or ill-defined. The student has not been able to identify sub-features or has used inappropriate grouping.
3-4	The student has produced an appropriate outline for the development, abstracted to an appropriate level of detail. The major features are identified and has provided suitable information to allow a formal specification for them.
5	The student has produced a clear breakdown of all reasonable features to solve the problem. The grouping is thoughtful and allows easy translation into requirements. The level of detail in the breakdown is appropriate at all levels.

Section 3 - Program Specification (10 marks)

The Program Specification is the final document, against which your program will be tested. This document not only says what the program should (and in some cases should not) do, but also who can do it if there are different types of users. For example, some features should only be available to administrators. You are able to use the feature groupings from your Problem Breakdown Document to organise the requirements, so that they can be easily found. Each requirement must be explained in one simple sentence, so that any non-developer could understand it.

This document is what your application is going to be measured against. If you produce functionality that is not included in the specification you cannot receive any marks for it. This is because the tester cannot know how that feature is supposed to work.

All specification points must have a priority attached to them, revise the MoSCoW method if you are unsure. Again, this is mostly for the benefit of the tester; i.e. your software will not fail if a '*Could*' feature is missing, but will fail if a '*Must*' feature is. These priorities will guide your development effort later.

There must be ***at least*** five '*Must*' specification points in your document. This submission should be ***no more than*** four A4 sides.

Mark Scheme

Mark	Criteria
0-2	The student has produced an incomplete specification or in an inappropriate level of detail. The work could not be used by an external developer to create their software. Requirements do not have any business ordering (MoSCoW or similar).
3-5	The requirements document is rudimentary but includes all major and most minor features. An external developer would be able to use the document to create the software, but is left with a large number of choices over how each feature works - making testing against the specification difficult. Business ordering is appropriate but could be missing *Won't* items to limit scope.
6-8	The specification is complete and fit-for-purpose. There are areas for improvement such as the detail required to develop algorithms but could be used with additional guidance to create a suitable test plan.
9-10	The student has produced a clear, concise and testable specification. The business ordering is reasonable and negative scoping (*Won't* items) are included to prevent wasted development effort. Minimal developer involvement would be required to produce an appropriate test plan.

Section 4 - Algorithm Pseudo Code Listings (5 marks)

All applications will have some form of algorithm in them. Every sort, search, add and remove has an algorithm. This section allows you to demonstrate understanding of the processes that your software requires. For example, what is the process/algorithm that an application goes through to check a username is not taken during registration? There will be at least one aspect of your project that requires an algorithm.

Remember that Pseudo Code is written in plain English. If you find yourself including bits of C# code, stop and think about the process rather than the code. Quite often students will complete an algorithm but miss out the steps needed for when things go wrong. The more specific and conscientious you are in this step, the easier the code becomes.

For this section you must submit ***at least*** two Pseudo Code examples from your application which should occupy ***no more than*** two A4 sides. Each Pseudo Code example should correspond to a different algorithm in your program.

Mark Scheme

This scheme will be applied for each algorithm submitted, the student will be credited with the highest two marks from those submitted.

Mark	Criteria
0-1	The student has produced an incomplete algorithm, missed or ill-defined critical steps. Pseudo code is either in inappropriate detail, improper language or indistinguishable from code.
2-4	The major components of the algorithm are included in appropriately chosen pseudo code. There may be minor edge or corner cases that are missing or not properly considered. There may be more technical elements that are expressed in 'real' code rather than pseudo code.
5	The student has produced an excellent example of pseudo code and correctly implemented the desired algorithm. The definition would allow the algorithm to be implemented in any appropriate language with little difficulty.

Section 5 - Program Code Listings (40 marks)

The largest part of your portfolio will be the source code for the program itself. This part will be marked on levels of commenting, formatting and style, implementation of your algorithms and the technical level of using C# and the .NET APIs. You will also lose marks if the submitted code cannot be run by the assessors as submitted.

Your code will be submitted electronically through Blackboard only. You should include **the entire project folder** including solution files, so that the assessors can run your project in Visual Studio.

Mark Scheme

Mark	Criteria
0-10	The student has produced a working prototype of the application. There may be features that are unimplemented, poor commenting and/or significant errors in the code. The student has not ventured beyond basic procedural C# and WPF/XAML.
11-20	The student's application is largely functional according to their specification. Code is commented appropriately, there may be minor bugs/errors. The student has attempted to use more advanced language features such as Events, Delegates and error handling.
21-30	The application is fully implemented according to their specification. Code is appropriately commented, structured appropriately and free from obvious (or statically identified) bugs/errors. The student has used more advanced language features correctly and has selected fitting support object/classes from the .NET APIs.
31-40	The student has produced an advanced, fully implemented, and elegant solution to the specification. Advanced language features and support object/classes have been used throughout. The student has implemented developer unit tests to ensure quality.

Section 6 - Test Plan for your Program (5 marks)

Students will be required to create a test plan for black-box testing their own finished application. This must cover all aspects in the specification whether the software contains that element or not. There will be a standard template available for all students to make use of. However, any student that decides not to use the template will not be penalised.

The plan will be marked on appropriateness (i.e. does the test actually exercise the functionality) and completeness (i.e. are all of the features exercised).

Mark Scheme

Mark	Criteria
0-1	The student has not formulated a plausible plan to exercise all features noted on the specification.
2	The student has formed a coherent plan that exercises most of the specification. Little to no detail is provided on how the tester should approach each test.
3-4	The student prepared an appropriate plan that exercises the specification of the program. Clear guidance is added for specific (and usually more technical) aspects. Prompts on what constitute a pass or fail at each point are provided.
5	The student has produced a formal test suite for the specification along with negative or destructive test cases. They have included strong guidance on how to complete each test and what constitutes a pass or fail for each.